1. Introduction

The Scientific Calculator project for CSE-115 is a console-based, menu-driven application. It is capable of performing a wide range of mathematical operations, elementary arithmetic to advanced scientific computations, including trigonometry, logarithms, power and root calculations, combinatorics, and matrix algebra. The project was developed collaboratively by a group of four members, each responsible for a distinct functional segment of the application.

Name	Segment
Khajwa Nawab Mohib	Arithmetic
Usayed Bin Gias	Trigonometric Operations
Abrar Shahriar	Logarithm, Factorial & Permutation/Combination
Augustine Gomes	Power & Root Operations, Matrix Operations

2. Objectives

Unlike standard calculators, this project emphasizes modular programming in C, like utilizing the switch-case system, function control, and “math.h” library to perform high-precision calculations. The calculator handles real-world constraints such as invalid input, domain violations, and undefined mathematical operations through systematic error checking. This report covers each segment and its system in detail, explaining the mathematical foundations, connecting with C code the implementation choices .

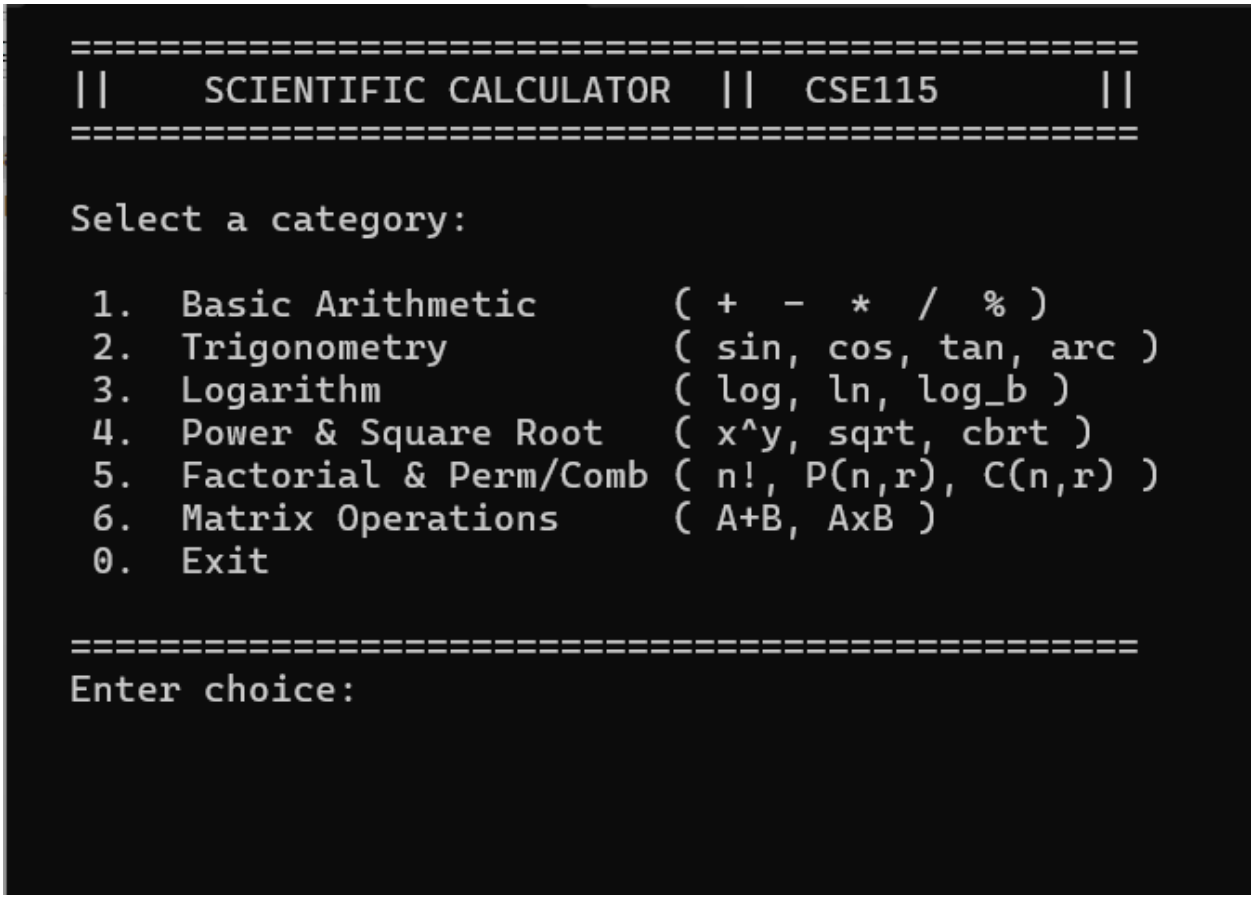
3. Tools and Environment

- Programming Language: C
- Compiler: GCC (GNU Compiler Collection) – gcc calculator.c -o calculator -lm
- Standard Libraries: stdio.h, math.h, stdlib.h
- Development Platform: Code Blocks - Windows

4. System Architecture

The user can selects a segment from 1–6, displayed in the menu function. Based on the Switch – Case system that corresponding function of that segment is been called. Within each sub-menu, the user selects a specific operation. After the computation completes, the program pauses and waits for the user to press ENTER before returning to the main menu. The loop continues until the user selects option 0 (Exit).

4.1 Console Menu



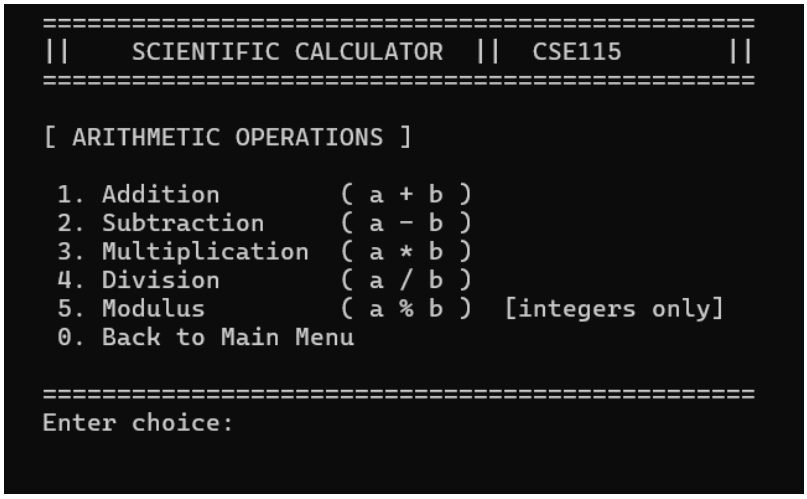
4.2 Standard C libraries:

- #include <stdio.h> – Standard input/output (printf, scanf)
- #include <math.h> – Mathematical functions (sin, cos, log, pow, sqrt, etc.)
- #include <stdlib.h> – System utilities (system() for screen clearing)

5. Arithmetic Operations

5.1 Operations and Sub-Segments

Arithmetic is the foundation of every calculator application. Khawja Nawab Mohib was assigned for implementing the five basic arithmetic operations: Addition, Subtraction, Multiplication, Division, and Modulus.

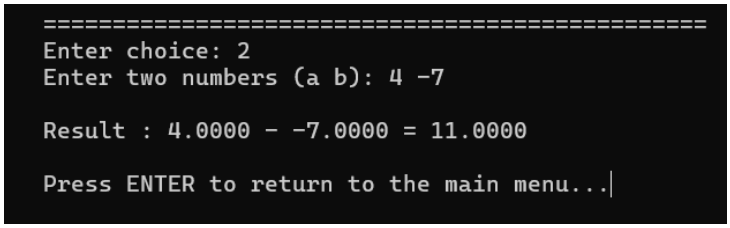


5.2 What the Functions Do

The menuArithmetic() function presents the user with a numbered sub-menu. After the user selects an operation, the function prompts for the required numeric inputs and computes the result using built-in C arithmetic operators. All operations except modulus work on double-precision floating-point values, ensuring decimal accuracy. Modulus is restricted to integers because the % operator in C is defined only for integer operands.

5.3 Addition, Subtraction, Multiplication, and Division

Upon selecting options 1 through 4, the program allows the user to enter two floating point numbers a and b stored as double. A switch statement routes control to the appropriate case, evaluates the expression, and prints the result formatted to four decimal places using the %.4lf format specifier. The use of double ensures that the calculator can handle values across a broad numerical range with approximately 15–16 decimal digits of precision.

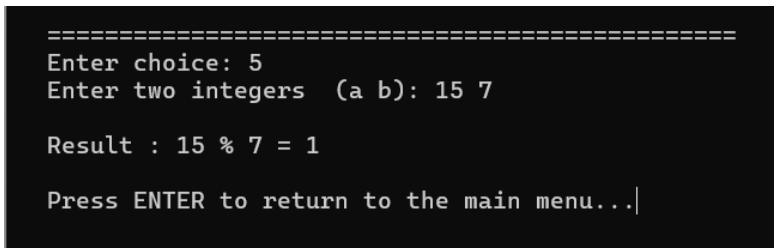


5.4 Code Snippet

```
case 1:
    result = a + b;
    printf("\n  Result : %.4lf + %.4lf = %.4lf\n", a, b, result);
    break;
case 4:
    if (b == 0) {
        printf("\n  [!] Error: Division by zero is undefined.\n");
    } else {
        result = a / b;
        printf("\n  Result : %.4lf / %.4lf = %.4lf\n", a, b, result);
    }
    break;
```

5.5 Modulus

The modulus operation (%) computes the remainder of integer division. Because the C modulus operator is defined only for integer operands, this operation handled separately. The two int variables “ia” and “ib” are declared and read via the “%d” format specifier

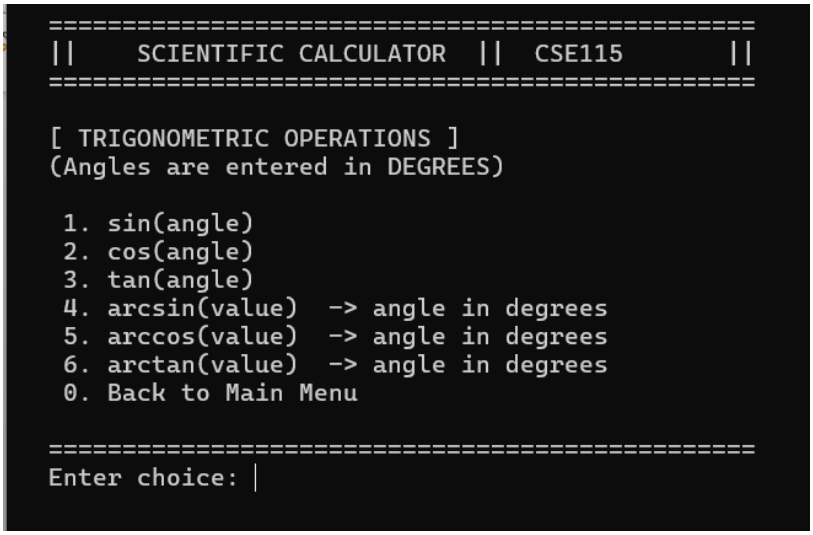


5.6 Error Handling in Arithmetic

i. Division by Zero (Case 4): Before performing division, the program checks if b == 0. If true, it prints “Error: Division by zero is undefined. and returns without computing.”
ii. Modulus by Zero (Case 5): Similarly, the modulus operation checks if ib == 0 before applying the % operator, preventing a runtime crash.
iii. Invalid Choice: If the user enters a menu number outside 0–5, an error message is displayed and the function returns without crashing.

6. Trigonometric Operations

The Trigonometry module, implemented in menuTrigonometry(), have six trigonometric functions with the three primary operation and their three inverses. Each of the six trigonometric operations accepts either an angle (for direct functions) or a ratio value (for inverse functions). Usayed Bin Gias was assigned to implement the following six trigonometric functions



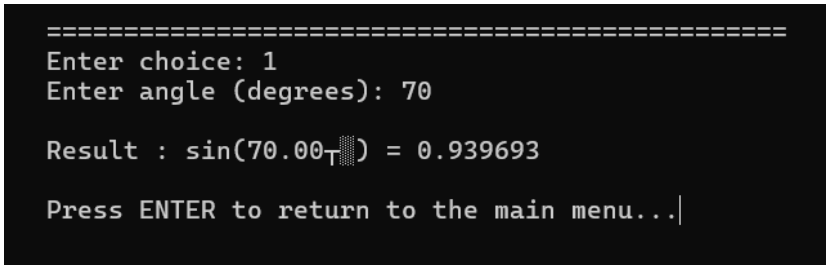
6.1 Degree - Radian Conversion

To make the calculator easy to use, they have the freedom to enter the values as Degree. Before computation the value is converted to Radian using the constant defined instruction - DEG_TO_RAD(x). Allowing the calculator to calculate the values more precisely. The constant defined, RAD_TO_DEG, converts the result to degree when necessary.

6.2 Sin, Cos, and Tan Functions

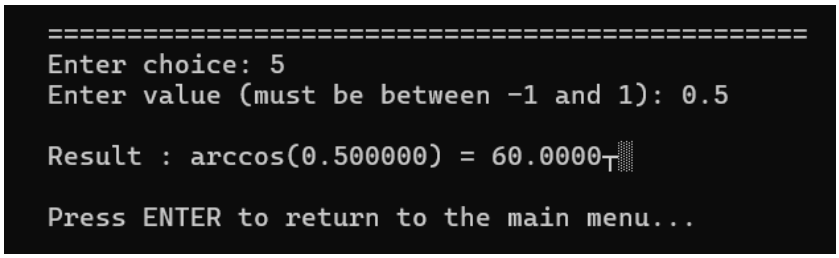
For cases 1 and 2, the user enters an angle in degrees. The value is converted to radians before passing to the C library functions sin() and cos() respectively. Both functions accept any real-valued angle, so no domain checking is required. Results are displayed to six decimal places, maintaining the precision.

The tangent function (case 3) requires special attention since it's a ratio of sin θ/ cos θ. Hence division by zero is undefined. According to the unit circle the cosine of an angle represents the x-coordinate. At 90° and 270° the x-coordinate is 0. So the implementation checks this condition using fmod() library function.



6.3 Inverse Trigonometric Functions

It takes a numeric value as input and return the corresponding angle in degrees. The arcsin and arccos have a restricted domain, their input must lie within the closed interval [−1, 1]. Whereas arctan (case 6) has no domain restriction since atan() is defined for all real numbers.



6.4 Error Handling in Trigonometry

i. tan(90°) Undefined: The program uses fmod(angle, 180.0) == 90.0 to detect this and prints an error.
ii. arcsin - arccos Domain Error: The program validates the input and rejects out-of
iii. range values [−1, 1] with a clear error message.

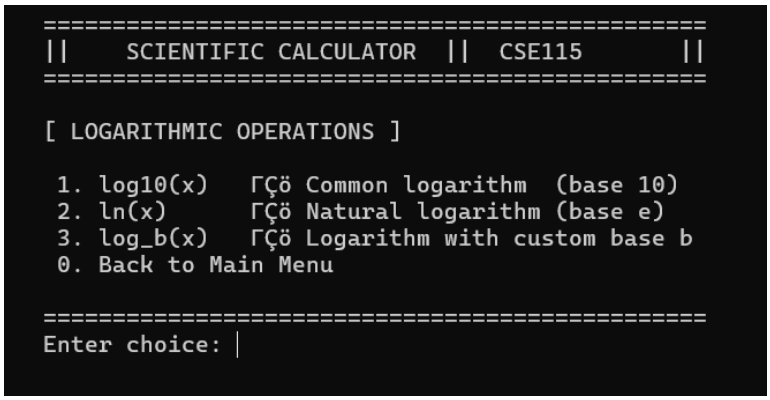
7. Logarithm, Factorial & Permutation/Combination

Shahriyar Abrar was assigned for two operations: Logarithmic Operations and Factorial & Permutation/Combination. These functions are implemented in menuLogarithm() and menuFactorialPerm() respectively. Both categories deal with operations that are heavily used in mathematics, statistics, information theory, and computer science.

7.1 Logarithmic Operations

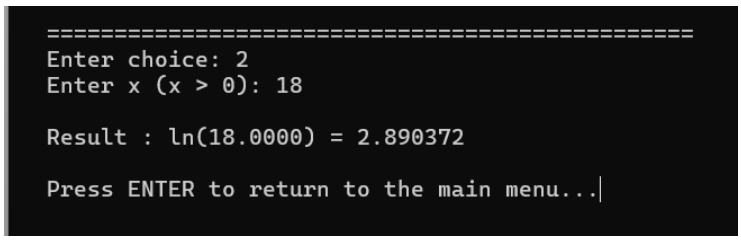
7.1.1 What the Function Do

Logarithm is considered as the inverse operation of exponential. The system allows to calculate three logarithmic modes: the common logarithm, the natural logarithm, and a general logarithm with a user-specified base. The operation is based on the C standard library functions $\log_{10}()$ and log() provided by math.h.



7.1.2 Logarithm Modes

The natural logarithm uses Euler's number e ≈ 2.71828 as its base. The common logarithm uses the log - base 10 directly from the library to compute. For the custom base b, the change-of-base formula is applied: $\log_b(x) = \ln(x) / \ln(b)$. Take the natural log of the argument and natural log of the base in lin(x) and ln(b) respectively.



7.1.3 Error Handling for Logarithms

i. Non-positive Input: Checks val <= 0 and prints an error before any computation.
ii. Invalid Base: For custom-base, the program checks that base > 0 and base != 1.

7.2 Combinatorics

7.2.1 What the Function Do

Its combinatorial section covering probability, statistics and discrete mathematics with the following operation: computing factorials (n!), permutations P(n, r), and combinations C(n, r). The three operations are accessed through the menuFactorialPerm() function.

```
================================================
||    SCIENTIFIC CALCULATOR  ||  CSE115       ||
================================================

[ FACTORIAL & PERMUTATION / COMBINATION ]

 1. Factorial     n!
 2. Permutation   P(n, r) = n! / (n-r)!
 3. Combination   C(n, r) = n! / (r! * (n-r)!)
 0. Back to Main Menu

================================================
Enter choice:
```

7.2.2 Factorial, Permutation & Combination

i) The factorial of a non-negative integer n, denoted n!, is the product of all positive integers from 1 to n, with the convention that 0! = 1. The helper function factorial() uses an iterative loop rather than recursion to avoid stack overflow for large values of n. The return type is double rather than int or long long to accommodate large factorials.

ii) Permutation P(n, r) counts the number of ways to arrange r items selected from n distinct items, where order matters. The formula is P(n, r) = n! / (n − r)! calling the factorial() function twice.

iii) Combination C(n, r) counts the number of ways to select r items from n distinct items where order does not matter. The formula is C(n, r) = n! / (r! × (n − r)!). It computes using three factorial() calls.

```
================================================
Enter choice: 1
Enter n (n >= 0): 6

Result : 6! = 720

Press ENTER to return to the main menu...
```

7.2.3 Error Handling for Combinatorics

i. Negative n: Factorial is undefined for negative integers. factorial() returns −1 if n< 0.
ii. Invalid Permutation/Combination Inputs: The program checks that r>= 0, n>= 0, and r<= n before computing. Not following these rules displays an error message.

7. Power, Root & Matrix Operations

Augustine Gomes was assigned to handle the two categories: Power & Root Operations and Matrix Operations. These operations can be selected respectively from menuPowerRoot() and menuMatrix(). Each functions have sub-shared menu that is functioned using the switch-case statement.

8. Power & Root Operations

8.1.1 What the Functions Do

The Power and Root module provides four operations: general exponentiation (x^y), square root ($\sqrt{x}$), cube root ($\sqrt[3]{x}$), and the nth root (x^(1/n)). The operations are handled using pow(), sqrt(), and cbrt() called from the math library. To maintain precision result formatted using the "%.6lf" format specifier.

```
================================================
||    SCIENTIFIC CALCULATOR  ||  CSE115       ||
================================================

[ POWER & ROOT OPERATIONS ]

 1. Power       x ^ y
 2. Square Root sqrt(x)
 3. Cube Root   cbrt(x)
 4. Nth Root    x ^ (1/n)
 0. Back to Main Menu

================================================
Enter choice:
```

8.1.2 General Power

The function takes input from user for base and power and stores them in variable "x" and "y" accordingly. Using the C standard library function pow(x, y) from math.h the value is calculated.

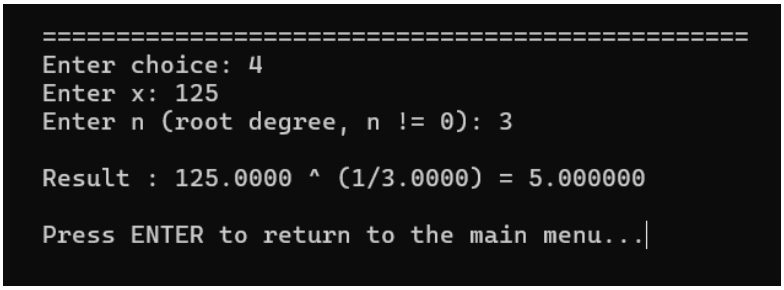
```
================================================
Enter choice: 1
Enter base x and exponent y: 4 3

Result : 4.0000 ^ 3.0000 = 64.000000

Press ENTER to return to the main menu...
```

8.1.3 Square Root, Cube Root, and Nth Root

The square root (case 2) takes input from the user for the double variable "x" and uses the library function sqrt(x).

The cube root (case 3) uses cbrt(x) from math.h. The difference only stands as the cube root is defined for negative real numbers.

The nth root (case 4) is the custom root operation. It is computed as pow(a, 1.0 / b). The 1.0 / b, the code ensures floating-point division is performed.



8.1.4 Error Handling for Power/Root

• Square Root of Negative: The program checks a < 0 before calling sqrt() for undefined for negative numbers

• Nth Root of Negative: The program uses fmod(b, 2.0) == 0 to detect degree of the roots and rejects negative inputs.

• Zero Root Degree: n = 0 would cause division by zero in 1/n. Explicitly rejected.

8.2 Matrix Operations

8.2.1 What the Functions Do

The menu menuMatrix() has shared sub-menu matrixAddition() and matrixMultiplication(), allowing two fundamental operations. A constant MAX_SIZE = 10 is defined to limit matrix dimensions with 2D arrays of type double[10][10]. Two functions handles the matrix input and display uniformly - readMatrix() allows the user to enter each element individually, using indexed prompts such as A[1][1], A[1][2], etc and printMatrix() displays a matrix in a neatly formatted grid with vertical bars as delimiters.

```
[ MATRIX OPERATIONS ]

 1. Matrix Addition         ( A + B )
 2. Matrix Multiplication   ( A x B )
 0. Back to Main Menu

================================================
Enter choice:
```

8.2.2 Matrix Addition – A + B

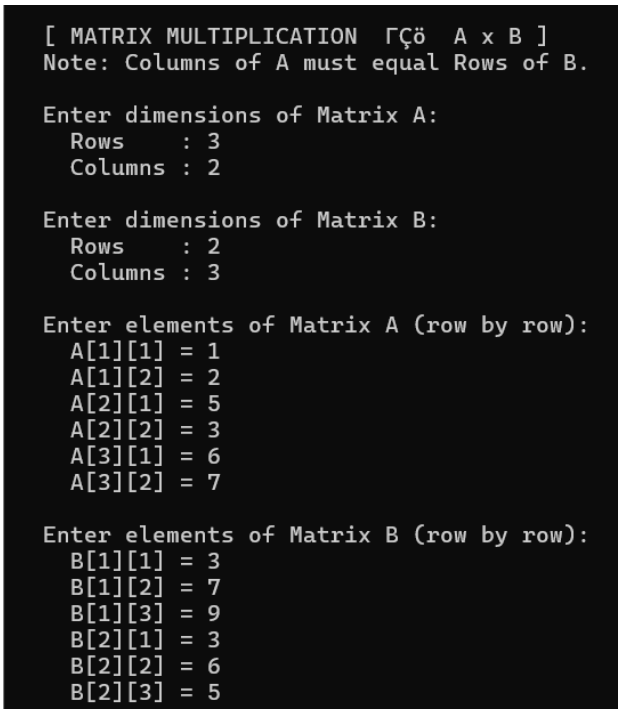
Matrix addition is defined only for matrices of identical dimensions. If A is an m×n matrix and B is a p×q matrix, A + B requires m = p and n = q. The formulation is represented by 2-d array C[i][j] = A[i][j] + B[i][j].

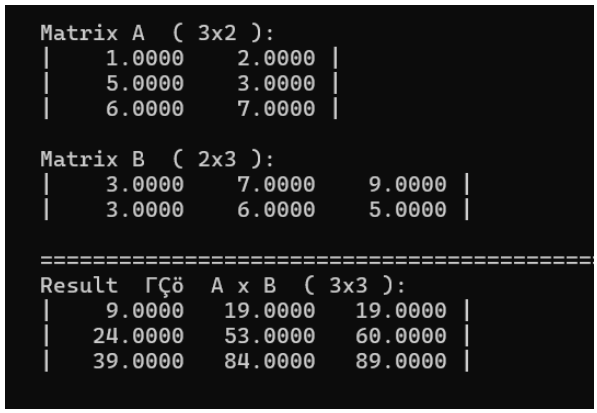
```
[ MATRIX ADDITION  ΓÇö  A + B ]

Enter dimensions of Matrix A:
  Rows    : 3
  Columns : 3

Enter dimensions of Matrix B:
  Rows    : 3
  Columns : 3

Enter elements of Matrix A (row by row):
  A[1][1] = 1
  A[1][2] = 3
  A[1][3] = 4
  A[2][1] = 3
  A[2][2] = 6
  A[2][3] = 7
  A[3][1] = 8
  A[3][2] = 9
  A[3][3] = 0

Enter elements of Matrix B (row by row):
  B[1][1] = 3
  B[1][2] = 2
  B[1][3] = 7
  B[2][1] = 2
  B[2][2] = 8
  B[2][3] = 6
  B[3][1] = 4
  B[3][2] = 3
  B[3][3] = 8
```

```
Matrix A:
|    1.0000    3.0000    4.0000 |
|    3.0000    6.0000    7.0000 |
|    8.0000    9.0000    0.0000 |

Matrix B:
|    3.0000    2.0000    7.0000 |
|    2.0000    8.0000    6.0000 |
|    4.0000    3.0000    8.0000 |

=======================================
Result  ΓÇö  A + B  ( 3x3 ):
|    4.0000    5.0000   11.0000 |
|    5.0000   14.0000   13.0000 |
|   12.0000   12.0000    8.0000 |
```

8.2.3 Matrix Multiplication – A × B

Matrix multiplication requires that the number of columns in A equals the number of rows in B. The product C = A × B is an m×n matrix where each element is computed as the dot product of the i-th row of A with the j-th column of B. The triple nested loop is implemented which is the classical matrix multiplication algorithm. The innermost statement C[i][j] += A[i][k] * B[k][j] accumulates the dot product element by element.





8.2.4 Error Handling for Matrix Operations

i. Dimension out of Range: The validDim() helper function checks both rows and columns must be between 1 and MAX_SIZE (10).

ii. Addition Incompatibility: Before addition, the program verifies that rA == rB and cA == cB. If not, it prints a detailed error showing both matrices' sizes.

iii. Multiplication Incompatibility: Before multiplication, the program checks cA == rB. If not, it states which dimension is mismatched and why the operation cannot proceed.